

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaier@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

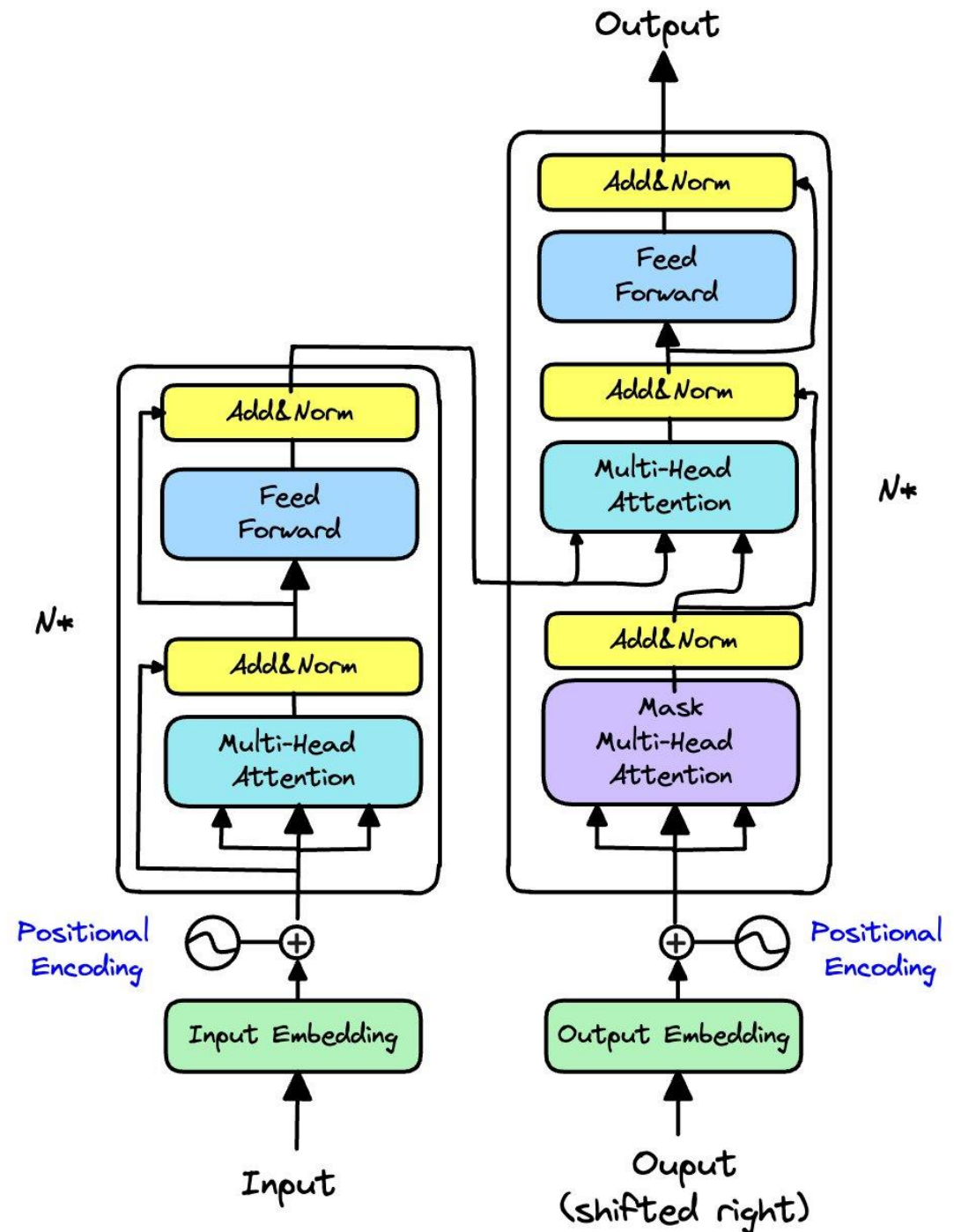
Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†]Work performed while at Google Brain.

[‡]Work performed while at Google Research.



```
class Transformer(nn.Module):
```

```
    def __init__(self, d_model=512, num_heads=8, num_encoders=6,
                  num_decoders=6, src_vocab_size=10000,
                  tgt_vocab_size=10000, max_len=5000):
```

```
        super().__init__()
```

```
        self.d_model = d_model
```

```
        # Encoder and Decoder
```

```
        self.encoder = Encoder(d_model, num_heads, num_encoders)
```

```
        self.decoder = Decoder(d_model, num_heads, num_decoders)
```

```
        # Positional encoding
```

```
        self.pos_encoding = PositionalEncoding(d_model, max_len)
```

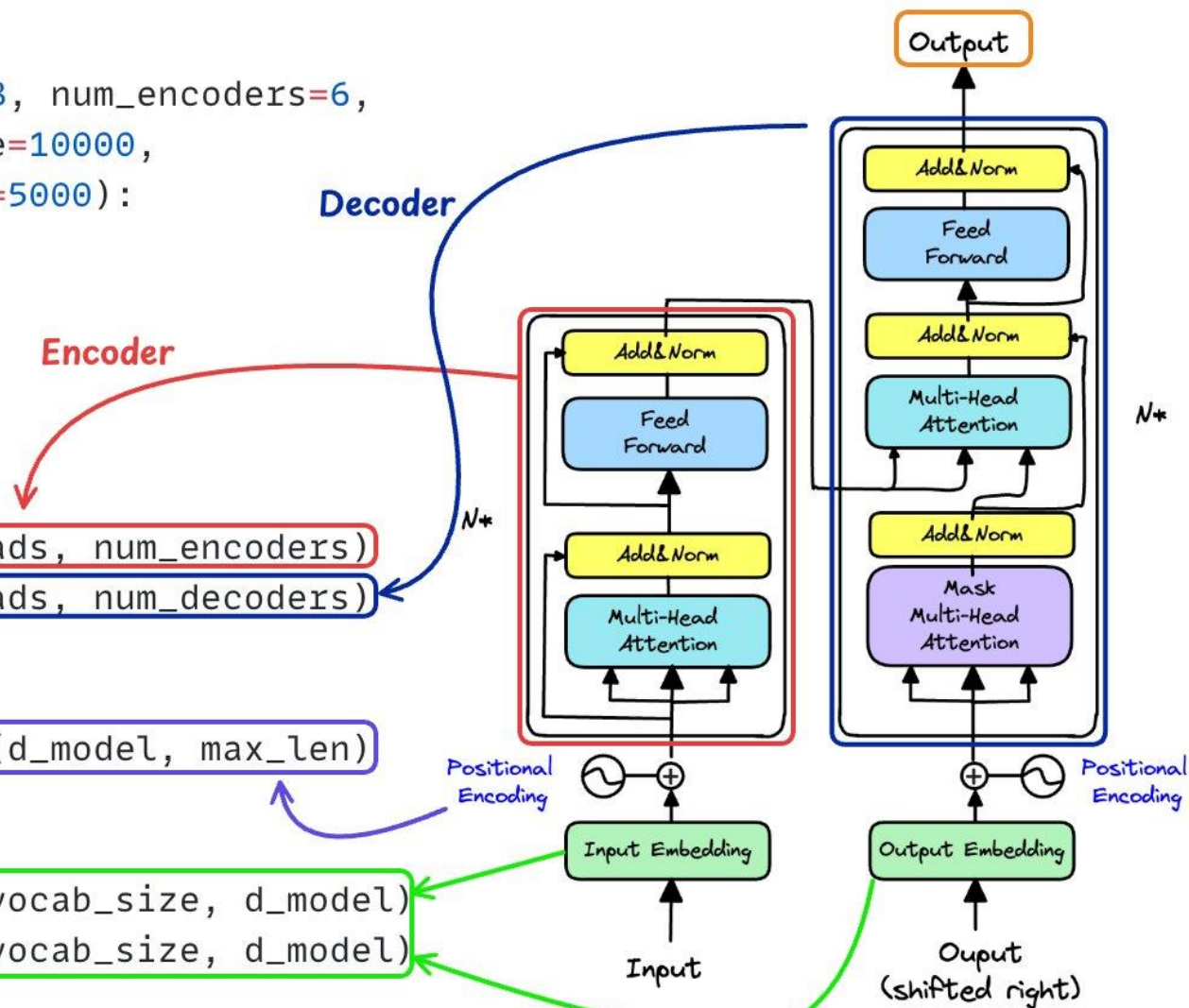
```
        # Embeddings
```

```
        self.src_embedding = nn.Embedding(src_vocab_size, d_model)
```

```
        self.tgt_embedding = nn.Embedding(tgt_vocab_size, d_model)
```

```
        # Output projection
```

```
        self.output = nn.Linear(d_model, tgt_vocab_size)
```



```

class Encoder(nn.Module):
    def __init__(self, d_model, num_heads, num_encoders):
        super().__init__()
        self.enc_layers = nn.ModuleList([
            EncoderLayer(d_model, num_heads) for _ in range(num_encoders)
        ])

```

Annotations: **Input Embedding dimension** points to `d_model`; **No. of attention heads** points to `num_heads`; **N = 6** points to `num_encoders`.

```

def forward(self, src, src_mask):
    output = src
    for layer in self.enc_layers:
        output = layer(output, src_mask)
    return output

```

```

class Decoder(nn.Module):
    def __init__(self, d_model, num_heads, num_decoders):
        super().__init__()
        self.dec_layers = nn.ModuleList([
            DecoderLayer(d_model, num_heads) for _ in range(num_decoders)
        ])

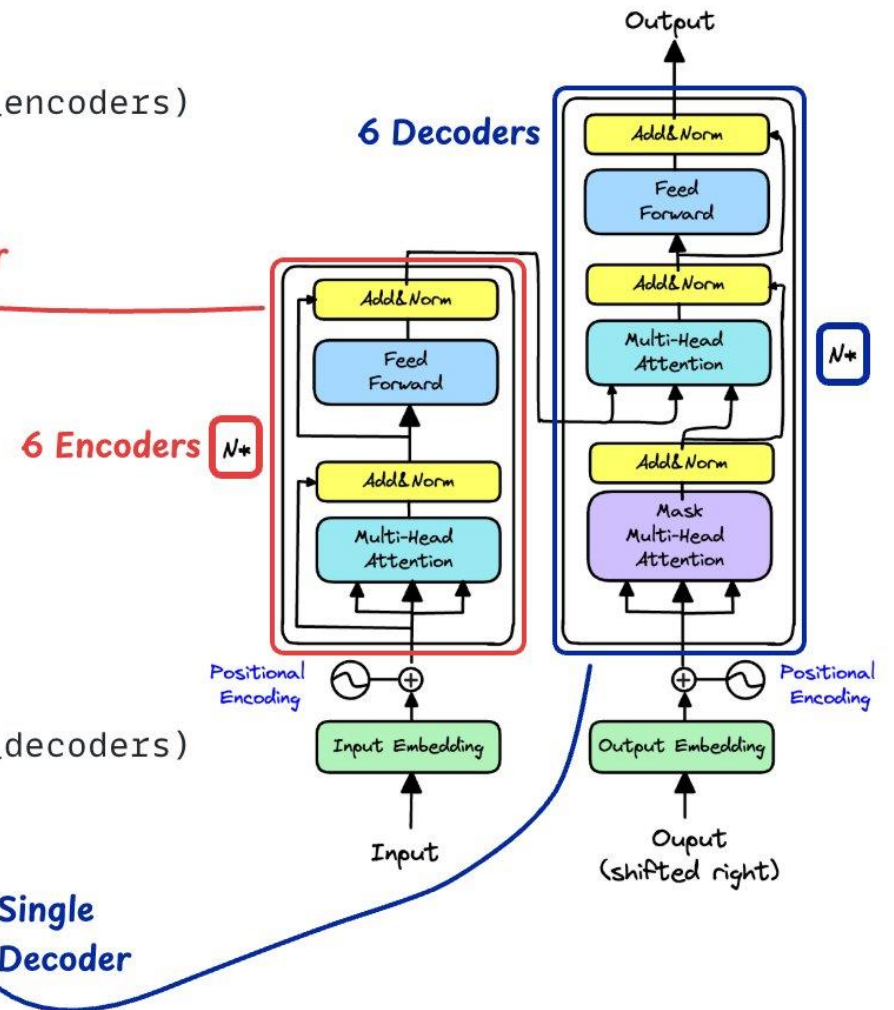
```

Annotation: **N = 6** points to `num_decoders`.

```

def forward(self, tgt, enc, tgt_mask, enc_mask):
    output = tgt
    for layer in self.dec_layers:
        output = layer(output, enc, tgt_mask, enc_mask)
    return output

```



```

class EncoderLayer(nn.Module):
    def __init__(self, d_model, num_heads, d_ff=2048, dropout=0.1):
        super().__init__()
        # attention
        ① self.attn = MultiHeadedAttention(d_model, num_heads, dropout)

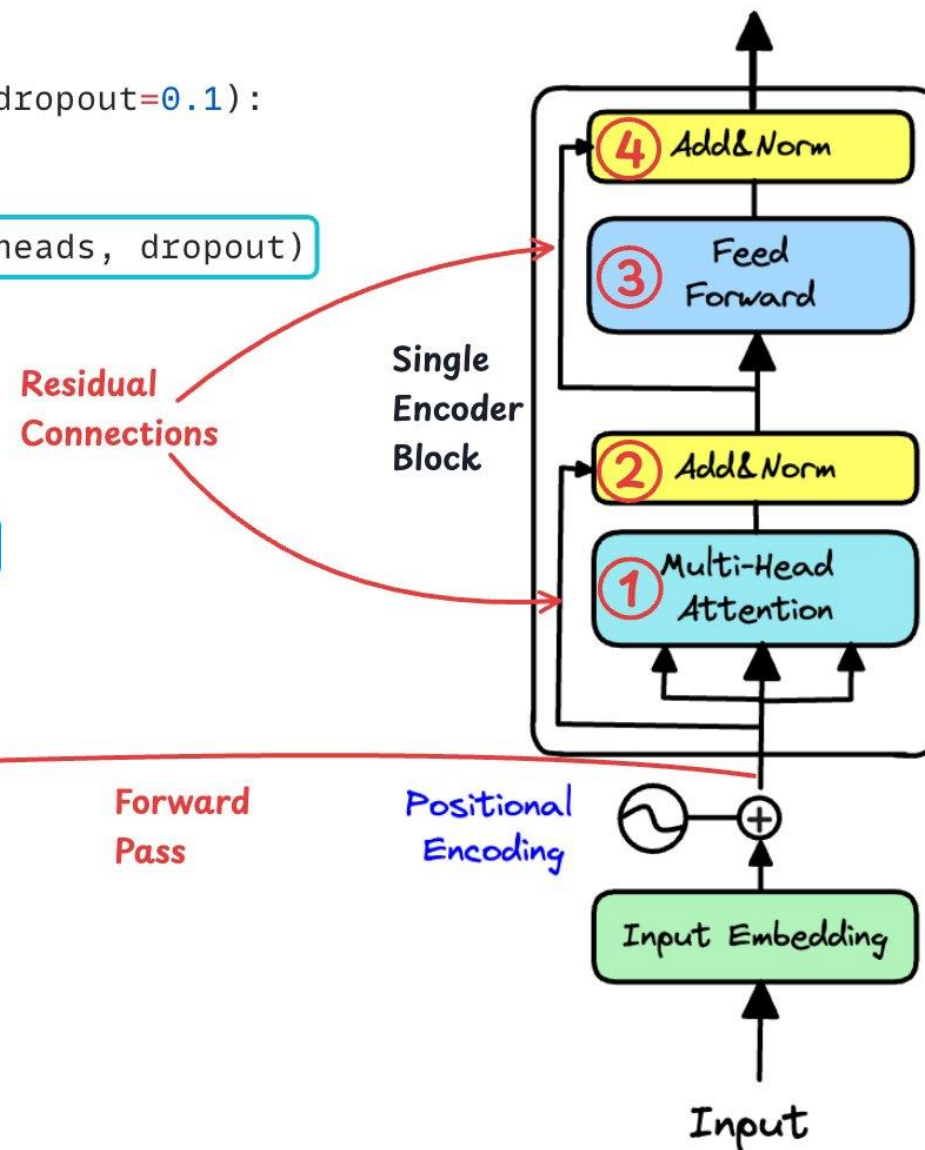
        # layer norm attention
        ② self.attn_norm = nn.LayerNorm(d_model)

        # ffn
        ③ self.ffn = FeedForward(d_model, d_ff, dropout)

        # layer norm ffn
        ④ self.ffn_norm = nn.LayerNorm(d_model)

    def forward(self, src, src_mask=None):
        # src
        x = src
        x = x + self.attn(x, x, x, mask=src_mask)
        x = self.attn_norm(x)
        x = x + self.ffn(x)
        x = self.ffn_norm(x)
        return x

```





output dim of
feedforward layer

```
class DecoderLayer(nn.Module):  
    def __init__(self, d_model, num_heads, d_ff=2048, dropout=0.1):  
        super().__init__()
```

① `self.masked_attn=MultiHeadedAttention(d_model,num_heads,dropout)`

② `self.masked_attn_norm = nn.LayerNorm(d_model)`

③ `self.attn = MultiHeadedAttention(d_model, num_heads, dropout)`

④ `self.attn_norm = nn.LayerNorm(d_model)`

⑤ `self.ffn = FeedForward(d_model, d_ff, dropout)`

⑥ `self.ffn_norm = nn.LayerNorm(d_model)`

```
def forward(self, tgt, tgt_mask=None, enc, enc_mask=None):
```

```
    x = tgt
```

```
    x = x + self.masked_attn(x, x, x, mask=tgt_mask)
```

```
    x = self.masked_attn_norm(x)
```

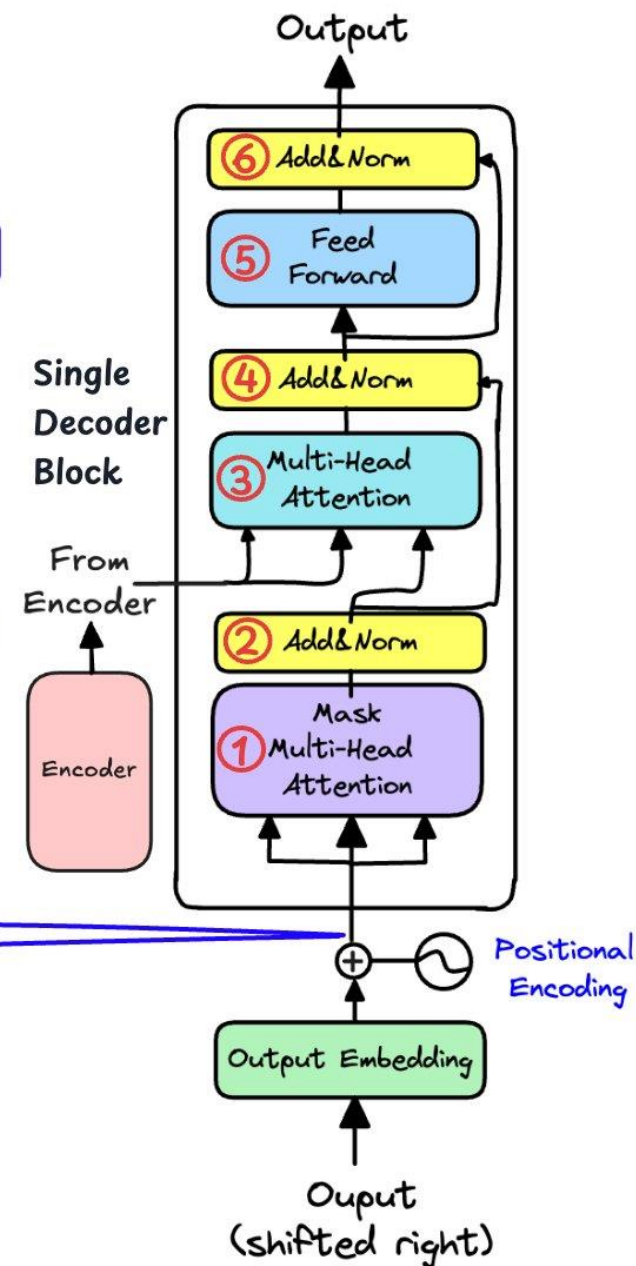
```
    x = x + self.attn(x, enc, enc, mask=enc_mask)
```

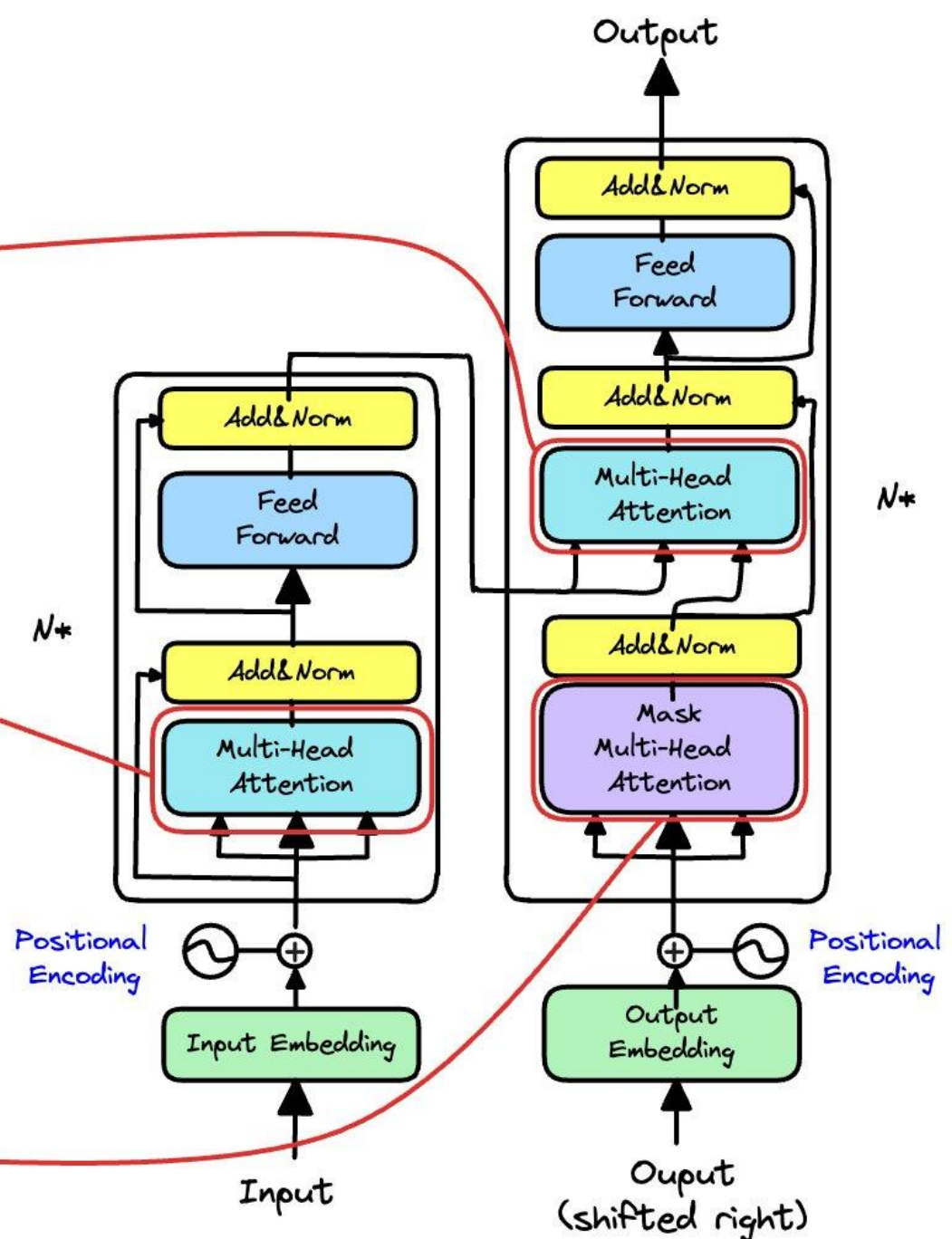
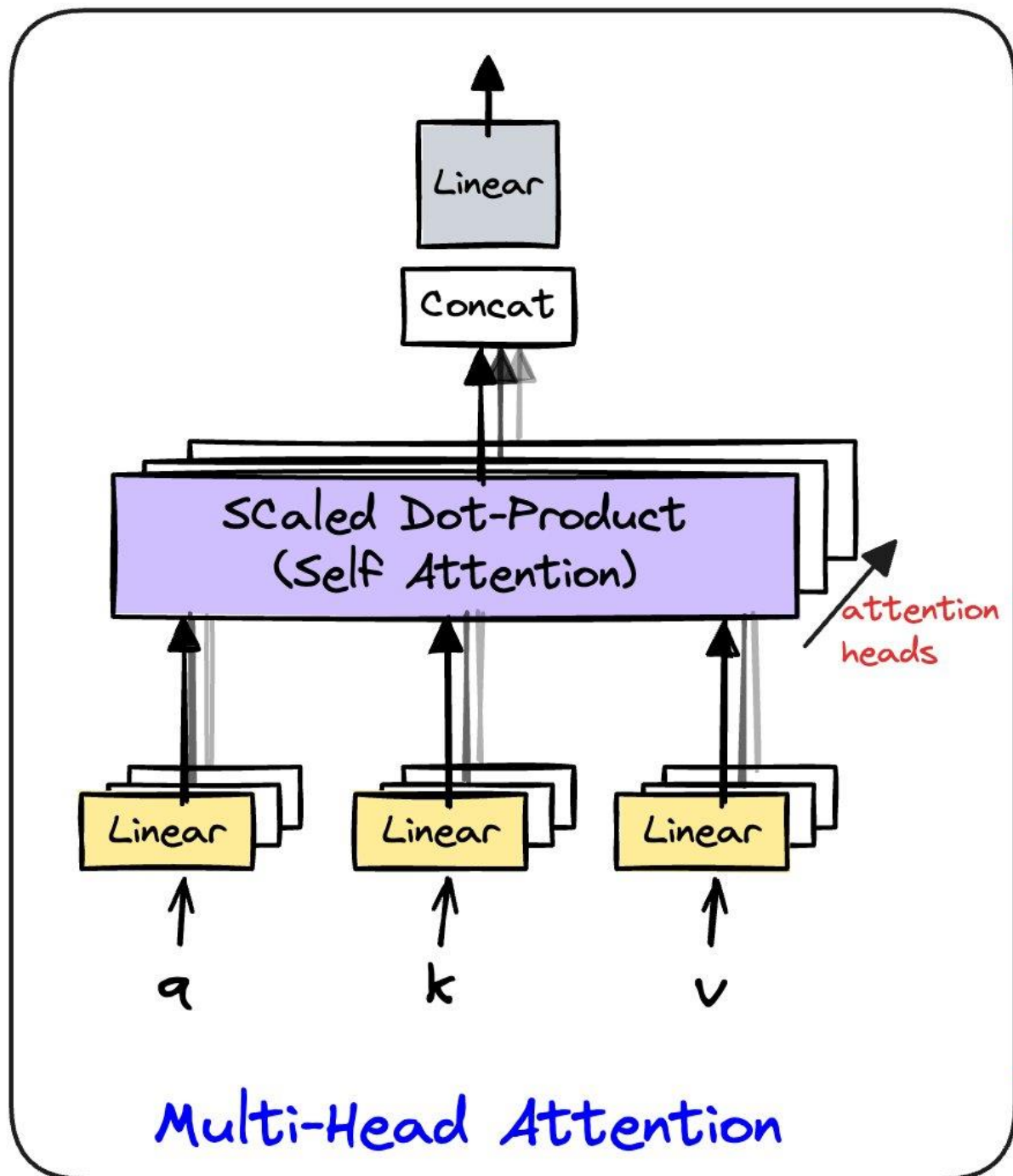
```
    x = self.attn_norm(x)
```

```
    x = x + self.ffn(x)
```

```
    x = self.ffn_norm(x)
```

```
    return x
```





```
class SelfAttention(nn.Module):
```

```
    def __init__(self, d_model, output_size, dropout=0.1):
        super().__init__()
        self.query = nn.Linear(d_model, output_size)
        self.key = nn.Linear(d_model, output_size)
        self.value = nn.Linear(d_model, output_size)
        self.dropout = nn.Dropout(dropout)
```

```
    def forward(self, q, k, v, mask=None):
```

```
        query = self.query(q)
        key = self.key(k)
        value = self.value(v)
```

Linear

Matmul

```
        dim_k = key.size(-1)
        scores = torch.bmm(query, key.transpose(1, 2)) / math.sqrt(dim_k)
```

```
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -float('inf'))
```

Mask(opt.)

```
        weights = F.softmax(scores, dim=-1)
```

Softmax

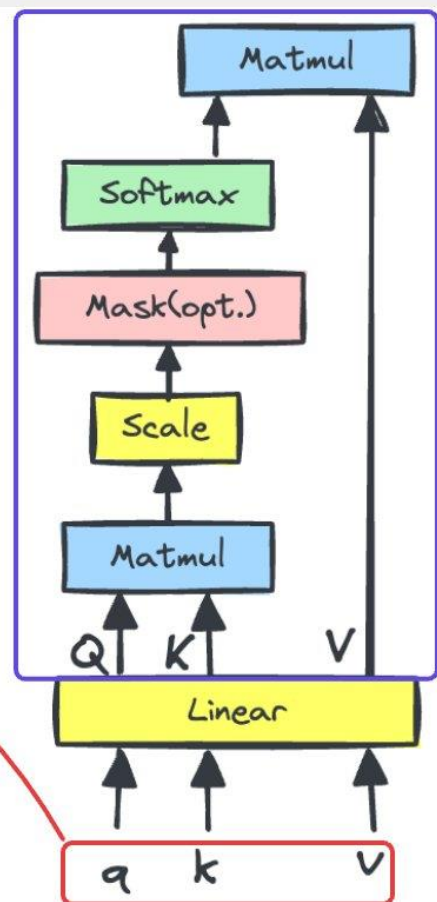
```
        weights = self.dropout(weights)
```

```
        outputs = torch.bmm(weights, value)
```

Matmul

```
        return outputs
```

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

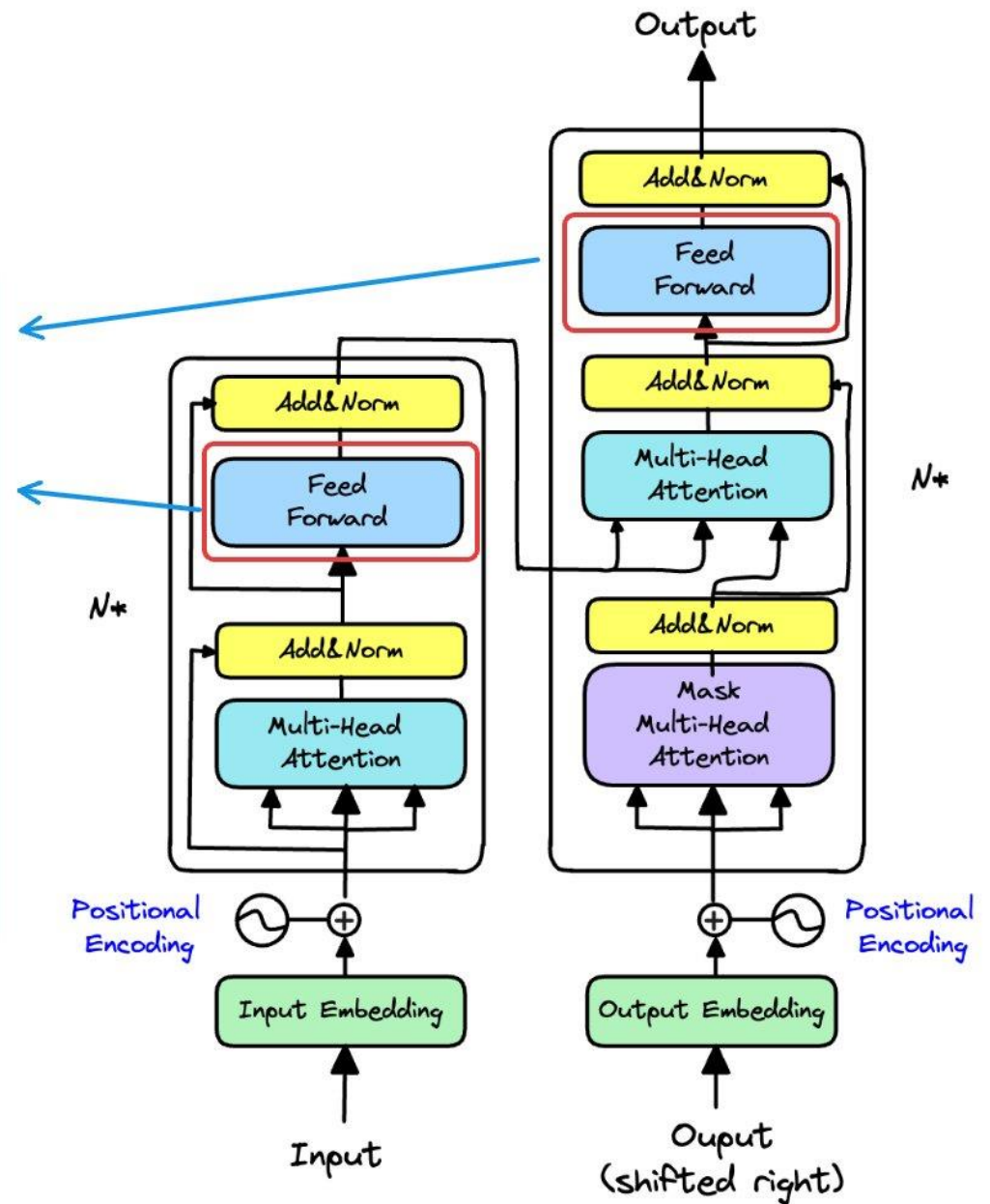


```

class FeedForward(nn.Module):
    def __init__(self, d_model, d_ff=2048, dropout=0.1):
        super().__init__()
        self.ffn = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.ReLU(inplace=True),
            nn.Dropout(dropout),
            nn.Linear(d_ff, d_model),
            nn.Dropout(dropout)
        )

    def forward(self, x):
        return self.ffn(x)

```



Sinusoidal functions for positional embeddings

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super().__init__()

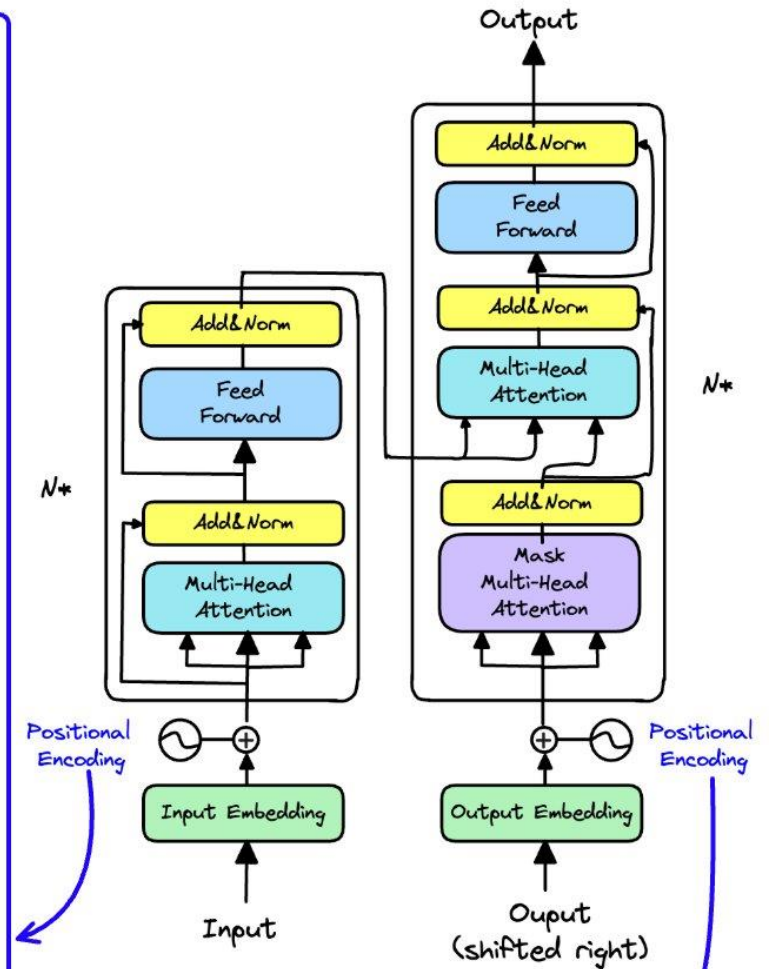
        pe = torch.zeros(max_len, d_model)

        position = torch.arange(
            0, max_len,
            dtype=torch.float
        ).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() *
                               (-math.log(10000.0) / d_model))

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

        self.register_buffer('pe', pe.unsqueeze(0))

    def forward(self, x):
        return x + self.pe[:, :x.size(1)]
```



```
class Transformer(nn.Module):
```

```
...
```

```
def forward(self, src, tgt, src_pad_token=0, tgt_pad_token=0):
```

```
    # Create masks
```

```
    src_mask = self.create_pad_mask(src, src_pad_token)
```

```
    tgt_mask = self.create_pad_mask(tgt, tgt_pad_token)
```

```
    subsequent_mask = self.create_subsequent_mask(tgt.size(1)).to(tgt.device)
```

```
    tgt_mask = tgt_mask & subsequent_mask
```

Token
Embeddings

```
    # Embeddings with scaling
```

```
    src_emb = self.src_embedding(src) * math.sqrt(self.d_model)
```

```
    tgt_emb = self.tgt_embedding(tgt) * math.sqrt(self.d_model)
```

Positional
Embeddings

```
    # Add positional encoding
```

```
    src_emb = self.pos_encoding(src_emb)
```

```
    tgt_emb = self.pos_encoding(tgt_emb)
```

Encoder

```
    # Encoder
```

```
    enc_out = self.encoder(src_emb, src_mask)
```

Decoder

```
    # Decoder
```

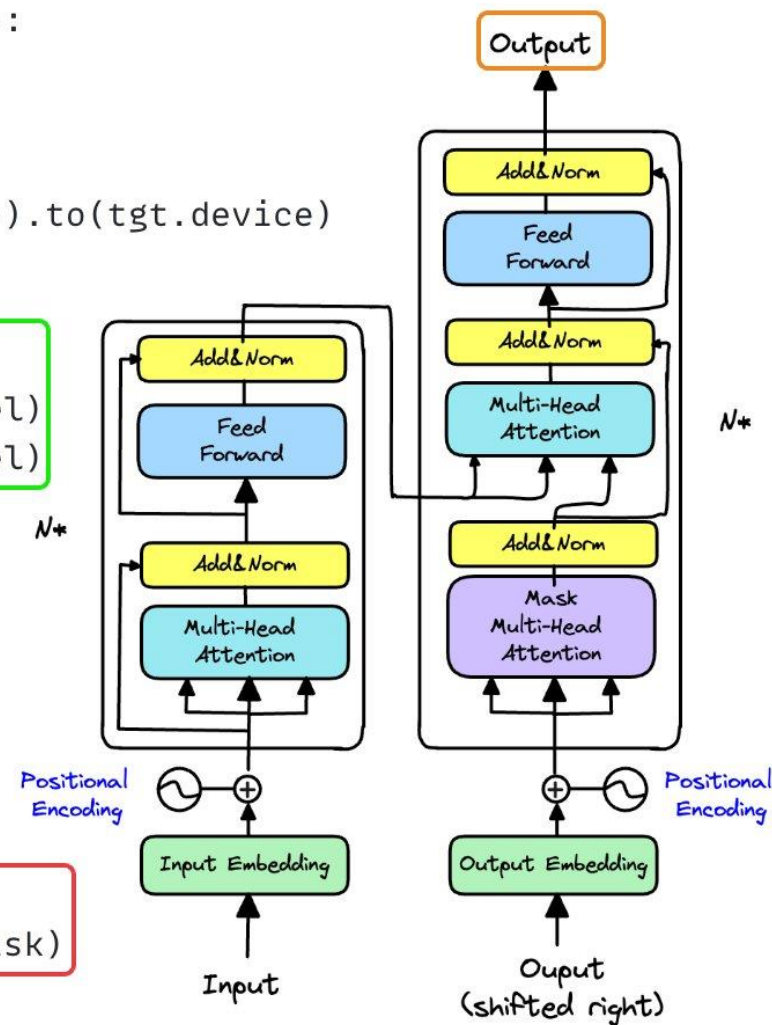
```
    dec_out = self.decoder(tgt_emb, enc_out, tgt_mask, src_mask)
```

Outputs

```
    # Output projection
```

```
    output = self.output(dec_out)
```

```
    return output
```



Transformer History

